DGSI-LAB4

Step 0 - Research previo (resumen corto)

En esta parte probé tres cosas: `sqlite3`, `subprocess.run()` y `wget`.

- SQLite: confirmé que todo vive en un archivo `.db`, no hay servidor separado.
- `subprocess.run()`: lo vi bloqueante (sincrono), y revisé `capture_output`, `text` y `timeout`.
- `wget`: con URL buena devuelve contenido, con URL mala devuelve error (codigo 4).

Evidencia:

- `evidence/step0_research.txt`

Step 1 - Single tool call

Implementé schema y función de `execute_sql`, y lancé un prompt para crear tabla. El modelo pidió la tool correctamente y la tabla se creó.

Fragmento de schema:

```
{
  "type": "function",
  "function": {
    "name": "execute_sql",
    "parameters": {
      "type": "object",
      "properties": {"query": {"type": "string"}},
      "required": ["query"]
    }
  }
}
```

Evidencia:

- `evidence/step1_single_tool_call.txt`

Step 2 - Loop con llamadas secuenciales

Añadí el bucle de rondas: mientras haya `tool_calls`, ejecutar tool y seguir. Parada cuando ya no hay tool call.

Fragmento del control:

```
if not message.tool_calls:
    print(message.content)
    break
```

Prueba hecha:

- crear tabla `cities`
- insertar 3 ciudades

Evidencia:

- `evidence/step2_loop_sequence.txt`

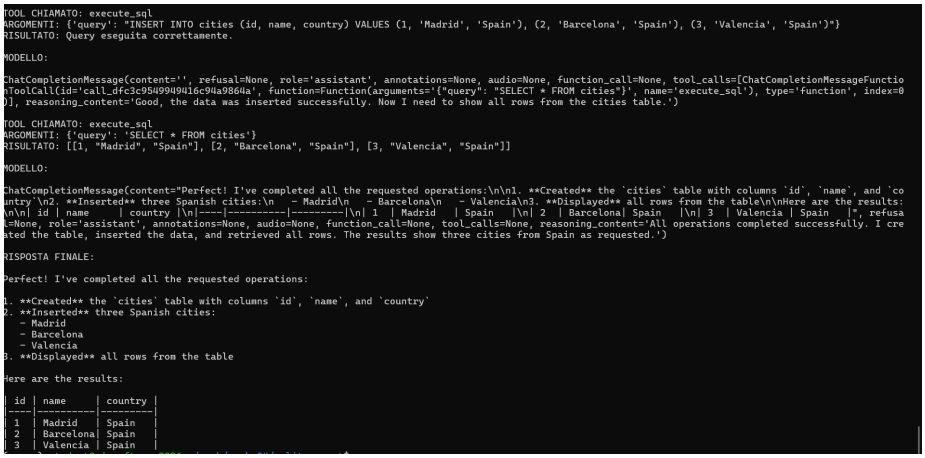

Figure 1: Loop delle iterazioni

Step 3 - wget con confirmacion humana

En este paso agregué confirmación antes de ejecutar `wget`. Si se deniega, se devuelve mensaje al modelo y no se ejecuta comando.

Fragmento:

```
answer = input("Allow command? (y/n): ")
if answer != "y":
    return "USER DENIED: command was not executed."
```

Evidencias:

- aprobado: `evidence/step3_wget_approve.txt`
- denegado: `evidence/step3_wget_deny.txt`

Figure 2: Prompt wget e richiesta di conferma

Step 4 - Test completo fetch + store + query

Prompt usado:

`Fetch https://jsonplaceholder.typicode.com/users and store the id, name, email, and city of every user in a SQLite table called users. Show me the final contents of the table.`

Resultado:

- se ejecutaron 5 iteraciones
- se descargaron usuarios
- se creó tabla
- se insertaron datos
- se consultó tabla final

Evidencias:

- run principal: `evidence/step4_full_test.txt`
- verificacion sqlite: `evidence/step4_sqlite_verify.txt`

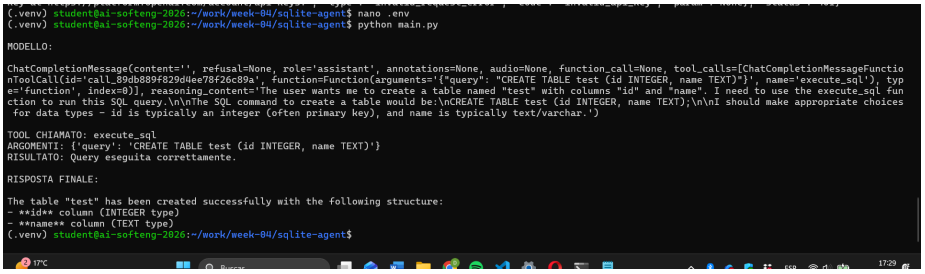

Figure 3: Schermata iniziale

Step 5 - Manejo de errores

Probé errores sin que el programa se cayera:

- URL inválida
- SQL inválido
- denegación manual de `wget`

Evidencias:

- `evidence/step5_bad_url.txt`

- evidence/step5_invalid_sql.txt
- evidence/step3_wget_deny.txt

```
MODELLO:

ChatCompletionMessage(content='Here\'s what happened:\n\n1. **JSON Download*
*: ❌ Failed - The URL download from https://jsonplaceholder.typicode.com/ab
cxyz returned a WGET ERROR (unknown wget error).\n\n2. **Table Creation**:
✅ Success - Created table `broken_users` with columns `id` and `name`.\n\n3
. **Invalid SQL Execution**: ❌ Failed - The query `INSRT INTO broken_users
VALUES (1, \'test\')` returned an SQL ERROR: near "INSRT": syntax error (you
 misspelled INSERT as INSRT).\n\n4. **Show All Rows**: ✅ Success - Query ex
ecuted, returning 0 rows: `[]`\n\nThe table is currently empty because the i
nvalid INSERT statement failed. Since we can\'t actually insert data (due to
 the intentional typo), there are no rows to display.', refusal=None, role='
assistant', annotations=None, audio=None, function_call=None, tool_calls=Non
e, reasoning_content='The table is empty because the INSERT statement failed
 due to the syntax error. Let me summarize all the results for the user.')

RISPOSTA FINALE:

Here's what happened:

1. **JSON Download**: ❌ Failed - The URL download from https://jsonplacehol
der.typicode.com/abcxyz returned a WGET ERROR (unknown wget error).

2. **Table Creation**: ✅ Success - Created table `broken_users` with column
s `id` and `name`.

3. **Invalid SQL Execution**: ❌ Failed - The query `INSRT INTO broken_users
 VALUES (1, 'test')` returned an SQL ERROR: near "INSRT": syntax error (you
misspelled INSERT as INSRT).

4. **Show All Rows**: ✅ Success - Query executed, returning 0 rows: `[]`

The table is currently empty because the invalid INSERT statement failed. Si
```

```
ARGOMENTI: {'url': 'https://jsonplaceholder.typicode.com/users'}
RISULTATO: USER DENIED

MODELLO:

ChatCompletionMessage(content='I apologize, but I encountered a "USER DENIED
" error when attempting to download the JSON from https://jsonplaceholder.ty
picode.com/users. This means I don\'t have permission to access that URL thr
ough my tools.\n\nUnfortunately, without being able to download the JSON dat
a first, I cannot proceed with creating the table and inserting the user rec
ords into the database. The workflow requires me to:\n1. Download the JSON c
ontent (which failed)\n2. Parse the data and create SQL statements\n3. Execu
te those SQL statements to create the table and insert data\n\nWould you lik
e me to try an alternative approach, or is there another URL I can use that
might be accessible?', refusal=None, role='assistant', annotations=None, aud
io=None, function_call=None, tool_calls=None, reasoning_content='The wget_ur
l tool returned "USER DENIED". This means I\'m not allowed to download conte
nt from that URL. I should explain this to the user clearly and let them kno
w I cannot complete the task as requested due to this restriction.')

RISPOSTA FINALE:

I apologize, but I encountered a "USER DENIED" error when attempting to down
load the JSON from https://jsonplaceholder.typicode.com/users. This means I
don't have permission to access that URL through my tools.

Unfortunately, without being able to download the JSON data first, I cannot
proceed with creating the table and inserting the user records into the data
base. The workflow requires me to:
1. Download the JSON content (which failed)
2. Parse the data and create SQL statements
3. Execute those SQL statements to create the table and insert data

Would you like me to try an alternative approach, or is there another URL I
can use that might be accessible?
(.venv) student@ai-softeng-2026:~/work/week-04/sqlite-agent$
```

Preguntas obligatorias

1. How does your program know when to stop calling the LLM?

Mi programa deja de llamar al modelo cuando la respuesta ya no trae ninguna petición de herramienta. Mientras el modelo sigue pidiendo acciones, el bucle continúa. En cambio, cuando devuelve una respuesta normal, sin pedir nada más, el programa entiende que esa ya es la respuesta final y ahí se para.

2. What is the role of tool_call_id in the message protocol?

Sirve para relacionar cada resultado con la llamada concreta que hizo antes el modelo. O sea, es como la forma de no mezclar una respuesta de una herramienta con otra. Gracias a eso, el sistema sabe exactamente qué resultado pertenece a qué petición.

3. Why is user confirmation important for wget but not for execute_sql?

Porque la descarga desde internet implica una acción externa y ahí tiene más sentido pedir permiso antes de ejecutarla. En cambio, la otra herramienta tra-

baja sobre la base de datos local de la práctica, así que estaba mucho más controlada y no tenía el mismo nivel de riesgo.

4. What happens in the conversation when the user denies a wget command?

Cuando el usuario no da permiso, la descarga no se ejecuta. Ese resultado se añade igualmente a la conversación y el modelo lo ve en la siguiente ronda. Entonces responde teniendo en cuenta que no pudo hacer esa parte, en lugar de inventarse un resultado como si la descarga hubiera funcionado.

5. How many iterations did the loop run for the full test prompt? Were you surprised?

En mi caso fueron cinco iteraciones. Primero hizo la descarga, luego creó la tabla, después insertó los datos, luego consultó el contenido y al final dio la respuesta final. La verdad es que no me sorprendió demasiado, porque viendo la tarea era bastante lógico que hiciera más o menos esos pasos.

Seguridad API key

Checklist cumplido:

- `.env` fuera de git
- `.env.example` presente
- `.gitignore` con `.env`, `database.db`, `__pycache__/`, `.venv/`

Repositorio

Link:

`https://github.com/danieledisciepolo/DGSI-LAB4`

Anexo A - Output completo del test principal

```
Week-04 Tool Loop Agent
Model: qwen3.5-122b-a10b
Endpoint: https://dashscope-intl.aliyuncs.com/compatible-mode/v1
Database: /tmp/week04/database.db
Confirm policy: always
Prompt: Fetch https://jsonplaceholder.typicode.com/users and store the id, name, email, and

=== Iteration 1 ===
[assistant] Requested 1 tool call(s)
[tool-call] id=call_939f4e38614b401ba2a4f6c3 name=wget
```

```
[tool-args] {"url": "https://jsonplaceholder.typicode.com/users"}
[confirm] LLM wants to run: wget -q -O - https://jsonplaceholder.typicode.com/users
[confirm] Auto-approval policy active: yes
[tool-result-preview] {"ok": true, "data": {"command": "wget -q -O - https://jsonplaceholder

=== Iteration 2 ===
[assistant] Requested 1 tool call(s)
[tool-call] id=call_be177700108a4924877055b4 name=execute_sql
[tool-args] {"query": "CREATE TABLE IF NOT EXISTS users (id INTEGER PRIMARY KEY, name TEXT,
[tool-result-preview] {"ok": true, "data": {"rowcount": -1, "lastrowid": 0}, "error": null}

=== Iteration 3 ===
[assistant] Requested 1 tool call(s)
[tool-call] id=call_2e3c409974a744bf88ce4616 name=execute_sql
[tool-args] {"query": "INSERT INTO users (id, name, email, city) VALUES (1, 'Leanne Graham'
[tool-result-preview] {"ok": true, "data": {"rowcount": 10, "lastrowid": 10}, "error": null

=== Iteration 4 ===
[assistant] Requested 1 tool call(s)
[tool-call] id=call_d0113c8b1b6648f49113563e name=execute_sql
[tool-args] {"query": "SELECT * FROM users"}
[tool-result-preview] {"ok": true, "data": {"columns": ["id", "name", "email", "city"], "row

=== Iteration 5 ===
[assistant-final]
I have successfully fetched the user data from the JSONPlaceholder API, created a SQLite tab

Here are the final contents of the `users` table:

| id | name | email | city |
| :--- | :--- | :--- | :--- |
| 1 | Leanne Graham | Sincere@april.biz | Gwenborough |
| 2 | Ervin Howell | Shanna@melissa.tv | Wisokyburgh |
| 3 | Clementine Bauch | Nathan@yesenia.net | McKenziehaven |
| 4 | Patricia Lebsack | Julianne.OConner@kory.org | South Elvis |
| 5 | Chelsey Dietrich | Lucio_Hettinger@annie.ca | Roscoeview |
| 6 | Mrs. Dennis Schulist | Karley_Dach@jasper.info | South Christy |
| 7 | Kurtis Weissnat | Telly.Hoeger@billy.biz | Howemouth |
| 8 | Nicholas Runolfsdottir V | Sherwood@rosamond.me | Aliyaview |
| 9 | Glenna Reichert | Chaim_McDermott@dana.io | Bartholomebury |
| 10 | Clementina DuBuque | Rey.Padberg@karina.biz | Lebsackbury |

[summary] iterations=5 final_text_preview=I have successfully fetched the user data from the
```

Anexo B - Verificacion sqlite independiente

```
1|Leanne Graham|Sincere@april.biz|Gwenborough
2|Ervin Howell|Shanna@melissa.tv|Wisokyburgh
3|Clementine Bauch|Nathan@yesenia.net|McKenziehaven
4|Patricia Lebsack|Julianne.OConner@kory.org|South Elvis
5|Chelsey Dietrich|Lucio_Hettinger@annie.ca|Roscoeview
6|Mrs. Dennis Schulist|Karley_Dach@jasper.info|South Christy
7|Kurtis Weissnat|Telly.Hoeger@billy.biz|Howemouth
8|Nicholas Runolfsdottir V|Sherwood@rosamond.me|Aliyaview
9|Glenna Reichert|Chaim_McDermott@dana.io|Bartholomebury
10|Clementina DuBuque|Rey.Padberg@karina.biz|Lebsackbury
```